MERN STACK

TRAINING TR-103 REPORT DAY 25

23 July 2026

Overview:

The twenty-fifth day of the MERN Stack training focused on developing a Login API using Node.js, Express.js, and MongoDB. We learned how to authenticate users by verifying their email and password, retrieve user data using the `findOne()` method, and send appropriate responses based on successful or failed authentication. The session also introduced the concept of user roles (`usertype`) for identifying different types of users after login.

Learning Objectives:

- Understand the working of a Login API.
- Learn user authentication using email and password.
- Retrieve user data using `findOne()`.
- Validate user credentials.
- Send appropriate success and error responses.
- Understand the use of `usertype` for role-based access.

Login API

A Login API is used to authenticate users by verifying the email and password entered during login. It checks whether the user exists in the database and validates the entered credentials before allowing access.

Features:

- Authenticates users.
- Verifies email and password.
- Allows only authorized users to log in.
- Returns user details after successful login.

POST Login Request

The client sends a POST request to the login endpoint with the user's email and password.

Syntax:

```
app.post("/api/login",  callback_function);
```

Features:

- Sends login credentials to the server.
- Uses the request body to receive user input.
- Initiates the authentication process.

findOne() Method

The findOne() method searches the database for a user whose email matches the email entered by the client.

Syntax:

```
usermodel.findOne({ Email: req.body.mail });
```

Features:

- Retrieves a single user document.
- Searches using the email field.
- Returns null if no matching user exists.

Password Validation

After finding the user, the server compares the stored password with the password entered by the user.

Features:

- Verifies user credentials.
- Prevents unauthorized access.
- Allows login only if passwords match.

Success Response

If authentication is successful, the server sends a response containing the user's details.

Response Includes:

- statuscode: 1
- Success message.
- User ID (user._id).
- User type (user.usertype).

Error Responses

If authentication fails, the server returns an appropriate error message.

Common Responses:

- Invalid Password – Password does not match.
- User Not Found – Email does not exist in the database.

Working of Login API

Steps:

1. Create a POST API using app.post("/api/login").
2. Search the database using findOne().
3. Check whether the user exists.
4. Compare the entered password with the stored password.
5. Return a success response if authentication is successful.
6. Otherwise, return an appropriate error message.

Task

```
app.post("/api/login",async(req,res)=>{
    const user=await usermodel.findOne({Email:req.body.mail});
    if(user){
        const fp=user.pass;
    if(fp==req.body.pass){
        res.send({statuscode:1,msg:"Login successful",id:user._id,usertype:req.body.usertype})
    }else{
        res.send({statuscode:0,msg:"Invalid credentials"})
    }
```

Conclusion:

By the end of Day 25, I gained a clear understanding of implementing a Login API in a MERN Stack application. I learned how to authenticate users by validating their email and password, retrieve user information using `findOne()`, return appropriate success or error responses, and identify users through their `usertype`, strengthening my understanding of user authentication and backend API development.